

Motivation

Traditional: Dense Vector

- Query: one vector
- Doc: several chunks

Really not good at multi-hop
Context Length
Cost on index & Time Consuming

Early: ColBERT & ColPali **MVR**

- Token-level & Patch-level

More Precise
Multimodal-transferable

Too Finer: Victoria Hong Kong has many what types
of building?

Perhaps coarser? phrases



How to Decompose	Non-differentiable	KELDaR	Query Decomposer
How to eval	Expensive	LLM-Optimizer Training Algorithm	Performance-Oriented

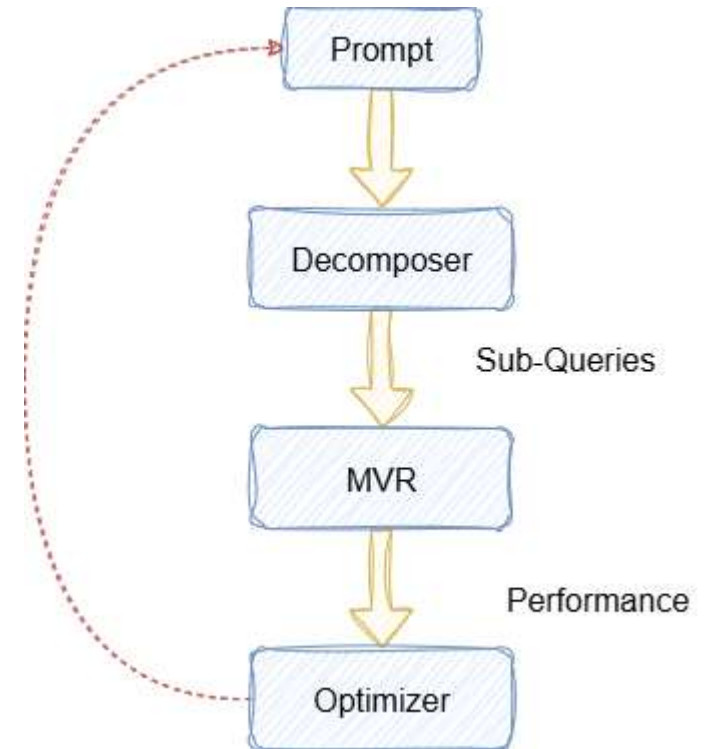
Approach

Sub-queries relies on **prompt** and **model parameters**

Sub-queries cannot be differentiated

Training model's parameters: θ and β

End2end



Approach

Algorithm 1

Goal: **generate&re-eval**

- Given the fixed Θ , generate new p_0 to construct p
- Decompose to sub-queries and compute loss

Input: Q^{train} , fixed Θ and p^{old}

Output: new prompt and its sub-queries

Data Structure: list(solution-score pairs)

1. $LS = [(p^{old}, \zeta(\Theta; p^{old}))]$
2. $Optimizer(2 \times \text{meta prompts} + \text{current } LS) \rightarrow p_0$
3. $p_0 + \text{task description} + q = p$
4. $Decomposer(p, q) \rightarrow q^{sub}$
5. $MVR(p) \rightarrow \zeta(\Theta; p)$
6. $LS.append(\zeta(\Theta; p))$

End condition: $\zeta(\Theta; p) - \zeta(\Theta; p_{old}) \leq \alpha$ OR k times

Algorithm 2

Goal: **alternation&incremental training**

- Joint Optimization among prompt and Θ
- Enable model to generate better in the downstream

Input: Q^{train} , generator θ and p^{old}

Output: new prompt and convergence θ

1. $Algo1(p^{old}, q) \rightarrow p^{new} \wedge q^{sub}$
2. If $p^{new} == p^{old}$, break
3. Using q^{sub} to train θ for τ epochs $\rightarrow \min(loss)$
4. $p^{new} = p^{old}$
5. Continue to train θ until convergence

End condition: $p^{new} == p^{old}$

Approach

Question 1: Why fix θ in algorithm 1?

Answer

- Optimize prompt rather than model
- Generalize to any frozen LLM for various diverse tasks
- $\min(\zeta(\Theta; p) - \zeta(\Theta; p_{old}))$
- If θ changes, improvement can't be isolated from prompt

Question 3: Why train θ with the fixed new prompt?

Answer

- Completely training θ in each epoch is expensive
- $\zeta(\theta^*(p^{old}); p^{old}) - \zeta(\theta^*(p^{new}); p^{new}) \geq \alpha - (1 - \frac{\mu}{2L})^\tau M$
- Incremental updates followed by final convergence training

Question 2: Why α doesn't guarantee decrease loss at convergence?

Answer

- α only guarantees loss decreases locally
- p^{new} may be good in few iterations, but until convergence, perhaps worse than p^{old}
- Set appropriate α and κ

Question 4: How to select α and τ ?

Answer

- α is too small, queries are over-decomposed
- α is too large, queries are rarely decomposed
- τ is too small, larger residuals weaken α 's gain
- τ is too large, smaller residuals slow convergence

Result Analysis

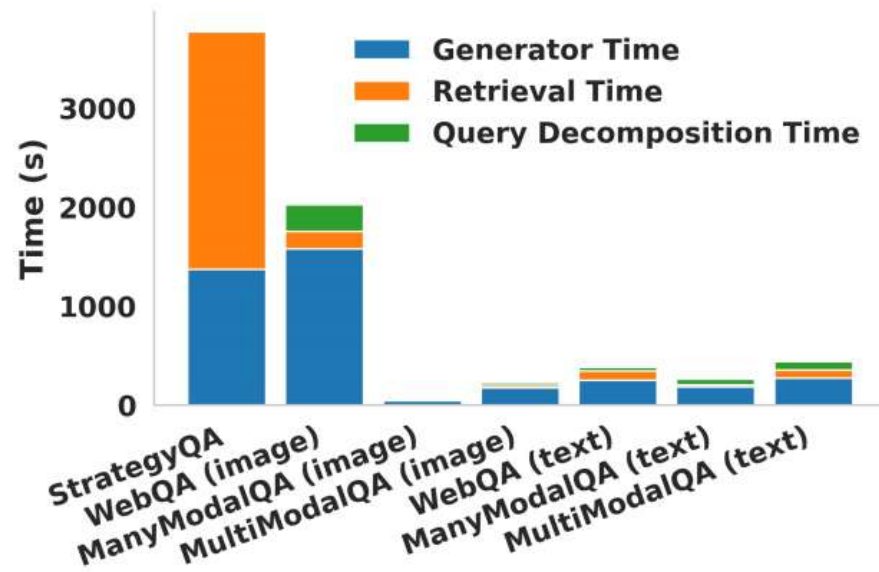


Figure 5. Overall inference time of POQD

POQD introduced minimal additional time overhead, aligning with the earlier part that it can be seamlessly integrated into RAG

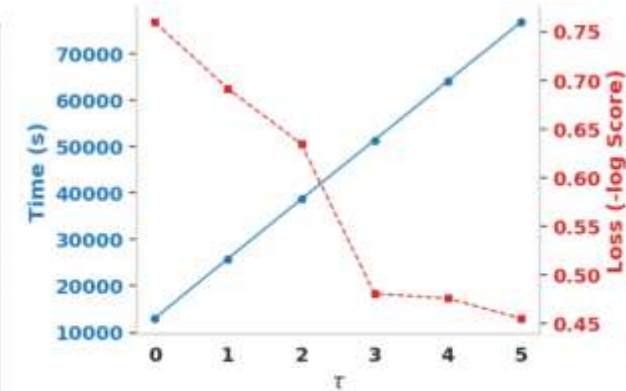
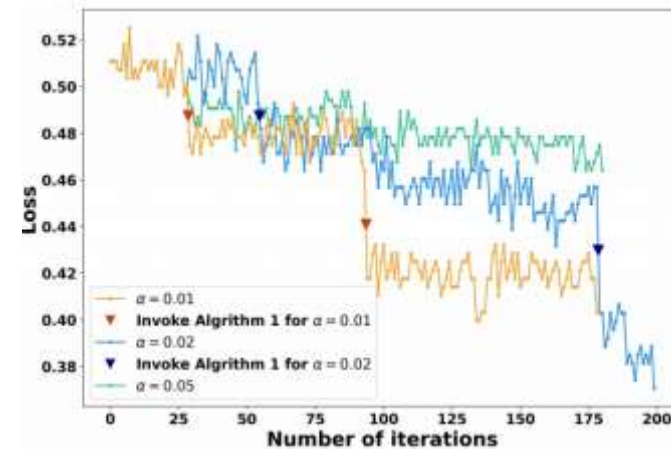


Figure 7. Training time of POQD with varied values of τ

- α is too big, underfitting
- α is too small, converge slowly
- $\tau = 3$, making balance between time and performance

Result Analysis

Dataset	<i>Image QA</i>			<i>Text QA</i>		
	WebQA	MultiModalQA	ManyModalQA	WebQA	MultiModalQA	ManyModalQA
	Hit@1	Hit@1	Hit@1	Hit@2	Hit@2	Hit@2
Dense retrieval	41.38	50.00	27.38	43.52	66.44	49.25
ColBERT	11.98	25.22	16.30	50.72	40.92	40.37
ColBERT-orig	38.95	36.96	21.05	<u>52.16</u>	<u>79.89</u>	<u>87.07</u>
S-QD	40.93	<u>52.61</u>	<u>28.15</u>	48.56	56.17	68.07
U-QD	33.89	51.74	26.86	46.04	45.21	67.89
ICL-QD	39.39	54.34	27.76	41.37	71.43	85.14
ICLF-QD	<u>41.69</u>	51.74	27.89	51.80	69.76	66.49
POQD	42.33	58.26	28.67	53.24	80.58	92.35

Table 1. Retrieval Accuracy on QA datasets

Dataset	<i>Image QA</i>			<i>Text QA</i>			
	WebQA	MultiModalQA	ManyModalQA	WebQA	MultiModalQA	ManyModalQA	StrategyQA
w/o RAG	80.31	16.52	30.98	56.47	40.36	32.28	58.76
Dense retrieval	81.43	46.09	34.12	59.35	59.36	41.25	61.43
ColBERT	82.14	42.61	29.80	60.79	53.95	22.08	62.45
ColBERT-orig	81.96	49.13	32.29	<u>61.14</u>	61.73	<u>77.66</u>	65.50
S-QD	81.73	<u>48.70</u>	<u>35.82</u>	58.99	54.92	62.62	<u>73.33</u>
U-QD	82.26	47.73	33.73	60.79	49.24	60.95	71.11
ICL-QD	<u>82.37</u>	46.78	34.70	58.63	61.86	76.69	60.70
ICLF-QD	81.54	46.55	34.77	60.42	<u>63.52</u>	60.07	72.34
POQD($\tau=3$)	82.83	61.74	37.92	62.22	68.10	81.27	75.55

Table 2. End-to-End QA (Exact Match) Accuracy. We bold the best and underline the second best accuracy number respectively

Overall SOTA, but:

1. Not good in high-noise and cross-modal reference settings
2. Underperforms on short-length queries and unstructured data

Sometimes, generator only can achieve good performance
Why don't set Hit@n on StrategyQA?

Pros and Cons

Pros

1. Decomposition costs few time, can be seamlessly integrated in RAG
2. The whole experiments are comprehensive and convincing
3. Coarser-grain matching is awesome
4. Theoretical analysis provides formal convergency guarantees
5. Interpretability and controllability, LLM as planner
6. Unified the framework for multimodal queries

Cons

1. Short on visual sub-queries examples
2. Sub-queries hallucination
3. Lacks Hit@n in StrategyQA task
4. Completely ignore the retriever parameter
5. Heavily rely on powerful LLM
6. Evals on Top-k and re-rank, more metrics